Trabalho ICD - grupo 5

Professora Ivette Luna

2024 Fortune 1000 Companies

1000 largest American companies maintained by Fortune magazine

Bianca Roma | RA: 213731

Giovanna Marcondes De Abreu | RA: 243896

Marcela Chenfel dos Anjos | RA: 159866

William Dadalt Makino | RA: 245811

24/11/2024

Fortune 1000 Companies: Data Frame 2024, 2023, 2022, 2021 e 2020

Índice	company	evenues_t	mployees_t	marketcap_t5	profits_t5	assets_t5	evenues_t	mployees_t	marketcap_t4	profits_t4	assets_t4	evenues_t	mployees_t	marketcap_t3	profits_t3	assets_t3	evenues_t	mployees_t	marketcap_t2	profits_t2	assets_t2
0	Walmart	648125	2.1e+06	484853	15511	252399	611289	2.1e+06	397475	11680	243197	572754	2.3e+06	409795	13673	244860	559151	2.3e+06	382643	13510	252496
1	Amazon	574785	1.525e+06	1.87368e+06	30425	527854	513983	1.541e+06	1.05844e+06	-2722	462675	469822	1.608e+06	1.65881e+06	33364	420549	386064	1.298e+06	1.55807e+06	21331	321195
2	Apple	383285	161000	2.64797e+06	96995	352583	394328	164000	2.60904e+06	99803	352755	365817	154000	2.84954e+06	94680	351002	274515	147000	2.05067e+06	57411	323888
3	UnitedHealth Group	371622	440000	456081	22381	273720	324162	400000	440854	20120	245705	287597	350000	479830	17285	212206	257141	330000	351725	15403	197289
4	Berkshire Hathaway	364482	396500	908920	96223	1.06998e+06	302089	383000	675657	-22819	948452	276094	372000	779542	89795	958784	245510	360000	587823	42521	873729
5	CVS Health	357776	259500	100374	8344	249728	322467	259500	95422	4149	228275	292111	258000	132839	7910	232999	268706	256500	98653.2	7179	230715
6	Exxon Mobil	344582	61500	461222	36010	376317	413680	62000	446424	55740	369067	285640	63000	349652	23040	338923	181502	72000	236355	-22440	332750
7	Alphabet	307394	182502	1.88463e+06	73795	402392	282836	190234	1.3302e+06	59972	365264	257637	156500	1.84233e+06	76033	359268	182527	135301	1.39256e+06	40269	319616
8	McKesson	276711	48000	70546.5	3560	62320	263966	66500	48757	1114	63298	238228	67500	45857.8	-4539	65015	231051	70000	31044	900	61247
9	Cencora	262173	44000	48472.1	1745.3	62558.7	238587	41500	32384	1699	56561	213989	40000	32355.7	1539.9	57337.8	189894	21500	24169.7	-3408.7	44274.8
10	Costco Wholesale	242290	316000	324924	6292	68994	226954	304000	220354	5844	64166	195929	288000	255231	5007	59268	166761	214500	155984	4002	55556
11	JPMorgan Chase	239425	309926	576938	49552	3.87539e+06	154792	293723	383549	37676	3.66574e+06	127202	271025	402527	48334	3.74357e+06	129503	255351	464531	29131	3.38607e+06
12	Microsoft	211915	221000	3.12613e+06	72361	411976	198270	221000	2.14605e+06	72738	364840	168088	181000	2.31136e+06	61271	333779	143015	163000	1.77823e+06	44281	301311
13	Cardinal Health	205012	47520	27217.8	261	43417	181364	46035	19452	-933	43878	162467	46827	15709.4	611	44453	152922	48000	17840.3	-3696	40766
14	Chevron	200949	45600	292966	21369	261632	246252	43846	311093	35465	257709	162465	42595	317120	15625	239535	94692	47736	201865	-5543	239790
15	Cigna	195265	71413	103018	5164	152761	180516	70231	75901	6668	143932	174078	72963	76286.3	5365	154889	160401	72963	83976.1	8458	155451
16	Ford Motor	176191	177000	53017.8	4347	273310	158057	173000	50400	-1981	255884	136341	183000	67958.2	17937	257035	127144	186000	48739	-1279	267261
17	Bank of America	171912	212985	299213	26515	3.18015e+06	115053	216823	228780	27528	3.05138e+06	93851	208248	332433	31978	3.1695e+06	93753	212505	333788	17894	2.81963e+06
18	General Motors	171842	163000	52353.5	10127	273064	156735	167000	51155	9934	264037	127004	157000	63555.2	10019	244718	122485	155000	82794.9	6427	235194
19	Elevance Health	171340	104900	120620	5987	108928	156595	102300	109098	6025	102772	138639	98200	118534	6104	97460	121867	83400	87908.9	4572	86615
20	Citigroup	156820	237925	121122	9228	2.41183e+06	101078	238104	91270	14845	2.41668e+06	79865	221768	105330	21952	2.29141e+06	88839	210153	151806	11047	2.26009e+06
21	Centene	153999	67700	41979.4	2702	84641	144547	74300	34810	1202	76870	125982	72500	49072.2	1347	78375	111115	71300	37169.6	1808	68719
22	Home Depot	152669	463100	380154	15143	76530	157403	471600	299534	17105	76445	151157	490600	309313	16433	71876	132110	504800	328775	12866	70581
23	Marathon Petroleum	150307	18200	72607.7	9681	85987	180012	17800	59544	14516	89904	141032	17700	47758.1	9738	85373	88952	57900	34873.8	-9826	85158
24	Kroger	150039	414000	41100.7	2164	50505	148258	430000	35421	2244	49623	137888	420000	41496.2	1655	49086	132498	465000	27064.3	2585	48662
25	Phillips 66	149890	14000	69880.8	7015	75501	175702	13000	46727	11024	76442	114852	14000	41561	1317	55594	65494	14300	35703.6	-3975	54721
26	Fannie Mae	141240	8100	1818.2	17408	4.32544e+06	121596	8000	475	12923	4.30529e+06	101543	7400	909.1	22176	4.22917e+06	106437	7700	2397.2	11805	3.98575e+06
27	Walgreens Boots Alliance	139081	268500	18712.3	-3080	96628	132703	262500	29836	4337	90124	148579	258500	38671.1	2542	81285	139537	277000	47455.3	456	87174
28	Valero Energy	139001	9897	56234.5	8835	63056	171189	9743	51351	11528	60982	108332	9804	41572.2	930	57888	60115	9964	29267.2	-1421	51774

795 empresas no total.

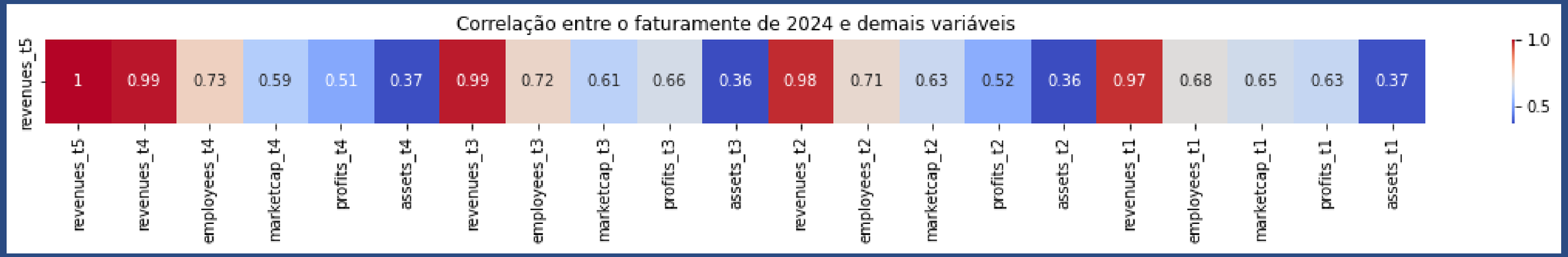
Separação dos dados entre conjunto de treinamento e de teste

```python
column_names = [
    'company',
    'revenues_t5',
    'revenues_t4',
    'employees_t4',
    'marketcap_t4',
    'profits_t4',
    'assets_t4',
    'revenues_t3',
    'employees_t3',
    'marketcap_t3',
    'profits_t3',
    'assets_t3',
    'revenues_t2',
    'employees_t2',
    'marketcap_t2',
    'profits_t2',
    'assets_t2',
    'revenues_t1',
    'employees_t1',
    'marketcap_t1',
    'profits_t1',
    'assets_t1'
    ]

input_vars = column_names[2:]

X = ft[input_vars] # variaveis independentes - features

y = ft['revenues_t5'] # variavel dependente - alvo

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 1234)
```

O nosso objetivo é a predição de uma série temporal. Utilizando os dados de faturamento, número de empregados, capitalização de mercado, lucros e ativos (em milhões de USD) de um intervalo de quatro anos, queremos prever o faturamento do ano seguinte.

```python
# t5 = 2024
# t4 = 2023
# t3 = 2022
# t2 = 2021
# t1 = 2020
```

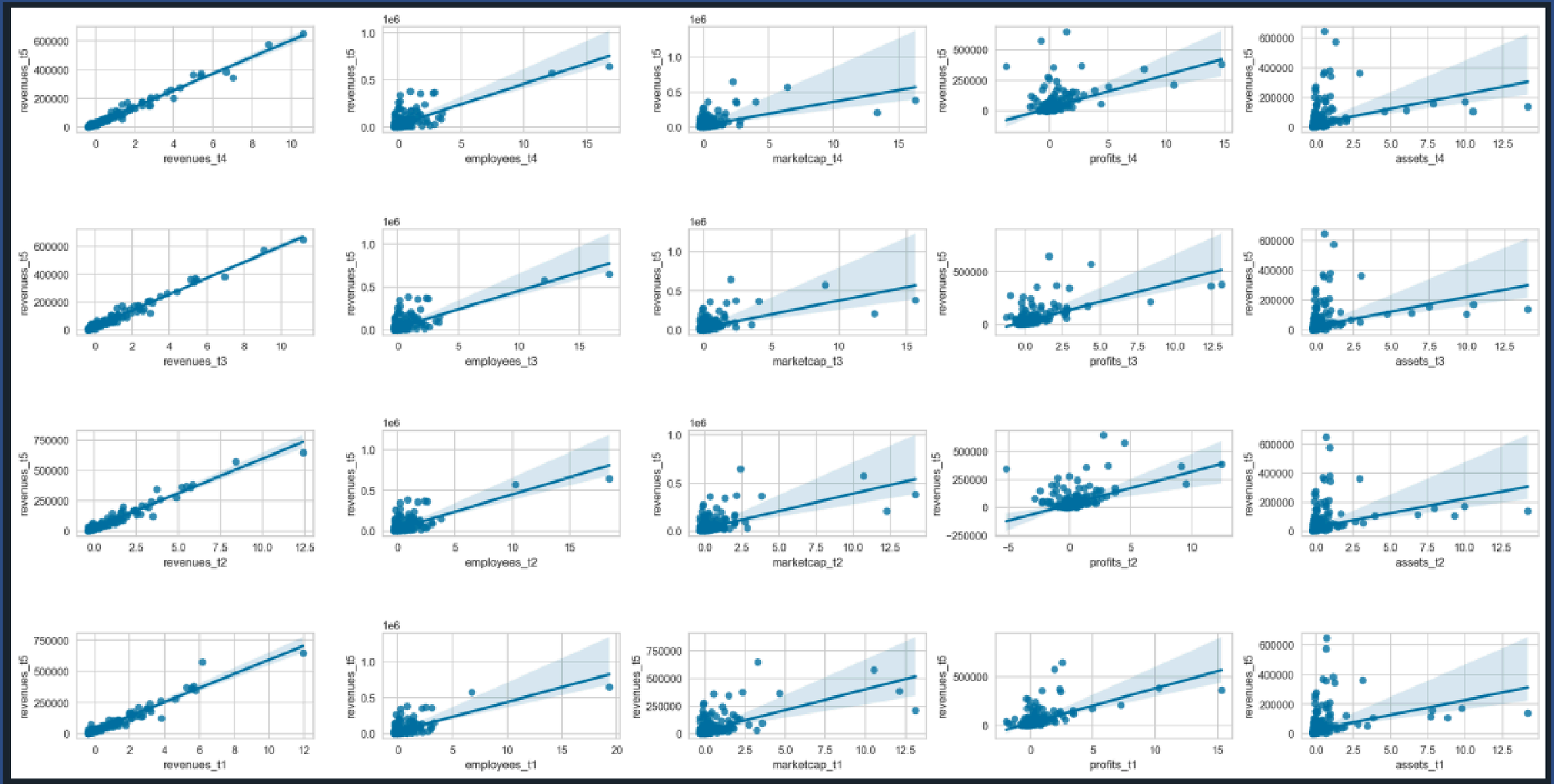
Correlação entre o faturamento de 2024 e as demais variáveis dos anos de 2023, 2022, 2021 e 2020



O problema da multicolinearidade na regressão linear:

- A regressão linear depende de um modelo matemático simples, onde cada variável tem uma contribuição clara e linear para a variável dependente.
- Quando as variáveis independentes estão fortemente correlacionadas, o modelo perde a capacidade de "distinguir" a contribuição de cada uma, o que leva a coeficientes inconsistentes e previsões erradas.
- Por isso, a multicolinearidade gera um problema diretamente relacionado com a precisão dos coeficientes da regressão, afetando a interpretação e a estabilidade das previsões.

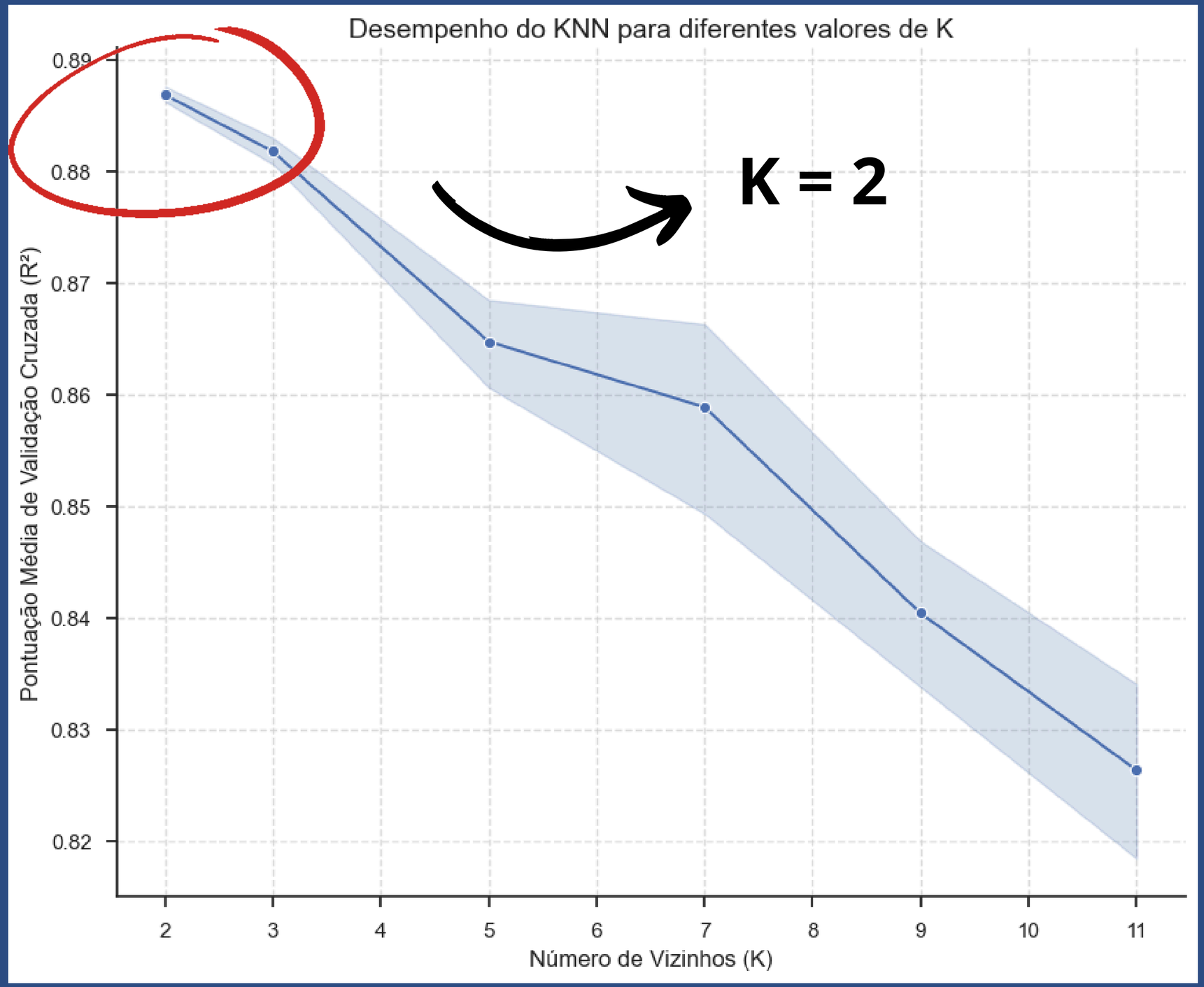
Gráficos de regressão entre as variáveis preditoras no eixo X e a variável alvo no eixo y



KNN Regressão

KNN: Grid Search e validação cruzada

```python
knn = KNeighborsRegressor(algorithm = 'auto') # algoritmo auto = escolhe automaticamente o método mais eficiente para encontrar os vizinhos

# definindo o grid de hiperparametros a ser buscado

knn_params_grid = {
    'n_neighbors': [2,3, 5, 7, 9, 11],  # numero de vizinhos
    'weights': ['uniform', 'distance'],  # tipo de ponderação
    'metric': ['euclidean', 'manhattan', 'minkowski']  # metrica de distancia
}

# Grid Search com validacao cruzada - Tunning

kf = KFold(n_splits = 20, shuffle = True, random_state = 42)

knn_gs = GridSearchCV(
    estimator = knn,       # modelo
    param_grid = knn_params_grid,  # grade de hiperparametros
    cv = kf,               # validacao cruzada com KFold
    scoring = 'r2',        # metrica de avaliação: R² (coeficiente de determinacao)
    n_jobs = -1,           # paralelizacao
    verbose =  1            # nivel de detalhamento
)

# treinando o modelo com os diferentes hiperparametros para chegar na melhor selecao de parametros

knn_gs.fit(X_train, y_train)

# melhores hiperparametros encontrados pelo Grid Search

print("Melhores parâmetros encontrados:", knn_gs.best_params_)

# Melhores parâmetros encontrados: {'metric': 'manhattan', 'n_neighbors': 2, 'weights': 'uniform'}

# selecionando o modelo com os melhores parametros encontrados

best_knn = knn_gs.best_estimator_
```


Desempenho do KNN para diferentes valores de K
K = 2
Pontuação Média de Validação Cruzada (R²)
0.89
0.88
0.87
0.86
0.85
0.84
0.83
0.82
2
3
4
5
6
7
8
9
10
11
Número de Vizinhos (K)

KNN: predição com dados de treino e teste

```python
# fazendo a predicao no conjunto de treinamento
y_pred_train_knn = best_knn.predict(X_train)
# ---------------------------------------------------------------
# fazendo a predicao no conjunto de teste
y_pred_test_knn = best_knn.predict(X_test)
# ---------------------------------------------------------------
# avaliando o desempenho, coeficiente de determinacao (R²), do conjunto de treinamento
r2_train_knn = r2_score(y_train, y_pred_train_knn)
print(f"R² Score no conjunto de treinamento (Modelo KNN): {r2_train_knn:.4f}")
# o Coeficiente de Determinação (R²) do conjunto de treinamento do modelo de kNN foi de 97,44%
# ---------------------------------------------------------------
# avaliando o desempenho, coeficiente de determinacao (R²), do conjunto de teste
r2_test_knn = r2_score(y_test, y_pred_test_knn)
print(f"R² Score no conjunto de teste (Modelo KNN): {r2_test_knn:.4f}")
# o Coeficiente de Determinação (R²) do conjunto de treinamento do modelo de kNN foi de 90,68%
```

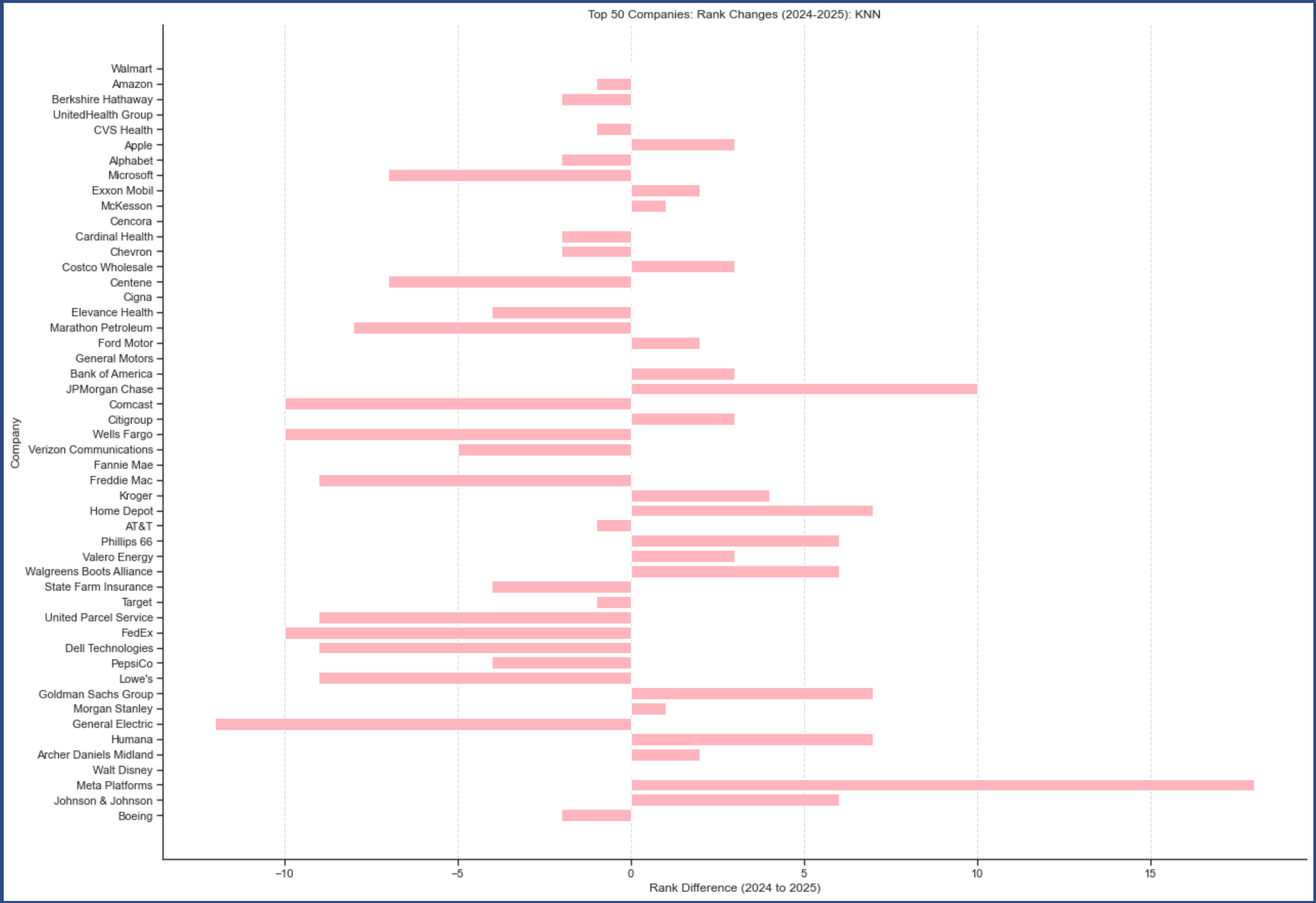
- Coeficiente de Determinação (R^2) do conjunto de treinamento do modelo de kNN foi de 97,44%
- Coeficiente de Determinação (R^2) do conjunto de teste do modelo de kNN foi de 90,68%

Prevendo ranking de 2025

```python
# inferencia com o modelo: KNN

# ----------------------------------------------------------------

y_2025_knn = best_knn.predict(X_inf)

# convertendo para DataFrame

df_2025_knn = pd.DataFrame(y_2025_knn)

df_2025_knn['company'] = ft_completo['company'] # adiciona a coluna 'company'

df_2025_knn = df_2025_knn.rename(columns = {0: 'revenues_2025'}) # renomeia a coluna do faturamento de 2025

# colocando a coluna 'company' a esquerda no DataFrame da coluna do faturamento predito de 2025

cols = ['company'] + [col for col in df_2025_knn.columns if col != 'company']

df_2025_knn = df_2025_knn[cols]

# criando a coluna de 'rank'

df_2025_knn['rank'] = df_2025_knn['revenues_2025'].rank(ascending = False, method = 'min')  # cria o rank com base em 'revenues_2025'

# colocando a coluna 'rank' mais a esquerda no DataFrame

cols = ['rank'] + [col for col in df_2025_knn.columns if col != 'rank']

df_2025_knn = df_2025_knn[cols]

# visualizando o DataFrame

print(df_2025_knn)
```

Ranking 2025

Índice	rank	company	revenues_2025	rank_difference	revenue_difference
0	1	Walmart	611455	0	-36670
1	1	Amazon	611455	-1	36670
4	3	Berkshire Hathaway	368052	-2	3570
3	4	UnitedHealth Group	364699	0	-6923
5	5	CVS Health	300033	-1	-57743
2	6	Apple	297600	3	-85685
7	6	Alphabet	297600	-2	-9794
12	6	Microsoft	297600	-7	85685
6	9	Exxon Mobil	286286	2	-58296.5
8	10	McKesson	269442	1	-7268.8
9	10	Cencora	269442	0	7268.8
13	12	Cardinal Health	233593	-2	28580.7
14	13	Chevron	198107	-2	-2842
10	14	Costco Wholesale	190686	3	-51604.5
21	15	Centene	188176	-7	34177
15	16	Cigna	183302	0	-11962.5
19	16	Elevance Health	183302	-4	11962.5
23	16	Marathon Petroleum	183302	-8	32995.5
16	19	Ford Motor	174016	2	-2174.5
18	19	General Motors	174016	0	2174.5
17	21	Bank of America	164366	3	-7546
11	22	JPMorgan Chase	156576	10	-82849
32	23	Comcast	146707	-10	25135
20	24	Citigroup	136080	3	-20740
33	24	Wells Fargo	136080	-10	20740
30	26	Verizon Communications	127773	-5	-6201
26	27	Fannie Mae	124645	0	-16595
35	27	Freddie Mac	124645	-9	16595
24	29	Kroger	123246	4	-26792.5


Top 50 Companies: Rank Changes (2024-2025): KNN
Company
Walmart
Amazon
Berkshire Hathaway
UnitedHealth Group
CVS Health
Apple
Alphabet
Microsoft
Exxon Mobil
McKesson
Cencora
Cardinal Health
Chevron
Costco Wholesale
Centene
Cigna
Elevance Health
Marathon Petroleum
Ford Motor
General Motors
Bank of America
JPMorgan Chase
Comcast
Citigroup
Wells Fargo
Verizon Communications
Fannie Mae
Freddie Mac
Kroger
Home Depot
AT&T
Phillips 66
Valero Energy
Walgreens Boots Alliance
State Farm Insurance
Target
United Parcel Service
FedEx
Dell Technologies
PepsiCo
Lowe's
Goldman Sachs Group
Morgan Stanley
General Electric
Humana
Archer Daniels Midland
Walt Disney
Meta Platforms
Johnson & Johnson
Boeing
−10
−5
0
5
10
15
Rank Difference (2024 to 2025)

SVR - Support Vector Regression

O SVR é um método que busca encontrar uma curva que melhor se ajuste aos dados, definindo uma "margem" chamada de tubo epsilon (ε), dentro da qual os erros não são penalizados.

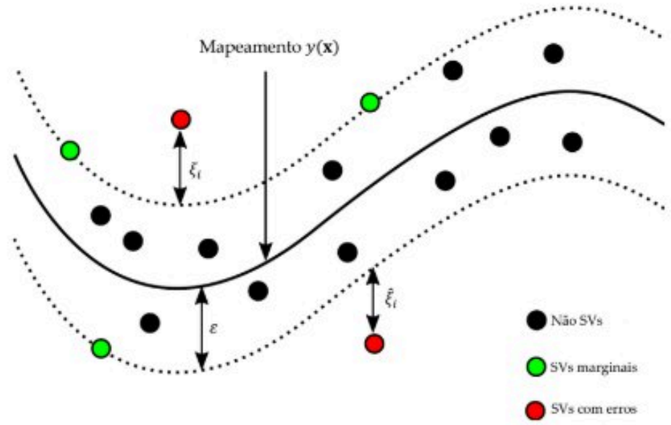


Figura 4 – Identificação dos vetores-suporte e das variáveis de relaxação, junto com a ilustração do mapeamento obtido por SVR. Adaptada de (SINGH ET AL., 2020).

- A <u>linha preta contínua</u> é a curva de regressão ajustada.
- Os **pontos pretos** (Não SVs) são os dados bem ajustado dentro do tubo ε.
- Os **pontos verdes** (SVs marginais) estão na borda do tubo, definindo os limites do modelo.
- Os **pontos vermelhos** (SVs com erro) estão fora do tubo e contribuem para o erro do modelo (com penalização)

```python
# ------------------------------------------------------------

# SVR - Support Vector Regression

# ------------------------------------------------------------

# definindo o modelo SVR

svr = SVR()

# ------------------------------------------------------------

# definindo o grid de hiperparametros a ser buscado

param_grid = {
    'kernel': ['linear'],  # tipo de kernel
    'C': [10, 50, 100, 500, 1000],           # parametro de regularizacao
    'epsilon': [0.01, 0.1, 0.2, 0.5, 1],  # parametro de margem
    'gamma': ['scale', 'auto']    # parametro do kernel RBF
}

# ------------------------------------------------------------

# Grid Search com validacao cruzada

grid_search = GridSearchCV(
    estimator = svr,
    param_grid = param_grid,
    cv = 10,  # numero de folds para validacao cruzada
    scoring = 'r2',  # metrica de avaliacao
    n_jobs = -1,  # paralelizacao
    verbose = 1
)

# ------------------------------------------------------------

# treinando o modelo com os diferentes hiperparametros para chegar na melhor selecao de parametros

grid_search.fit(X_train, y_train)

# ------------------------------------------------------------

# melhores hiperparametros encontrados pelo Grid Search

print("Melhores parâmetros encontrados:", grid_search.best_params_)

# C: parametro de regularizacao, que controla o trade-off entre a maximizacao da margem e a minimizacao do erro de treinamento -> 1000
# epsilon: determina a largura da faixa onde nao ha penalidade para erros, o que afeta a precisao da previsao -> 0.5
# gamma: controla a influencia de cada ponto de treinamento, como gamma = 'scale', o calculo é 1 / (n_features * X.var()) -> 'scale'
# kernel -> linear
```

SVR - Support Vector Regression

Depois de treinar o SVR com os melhores hiperparâmetros encontrados pelo Grid Search (busca em grade), calculamos o coeficiente de determinação (R^2) dos conjuntos de treinamento e de teste, que é a métrica estatística que avalia o desempenho do modelo.

```python
# selecionando o modelo com os melhores hiperparametros encontrados
best_svr = grid_search.best_estimator_
# ------------------------------------------------------------
# fazendo a predicao no conjunto de treinamento
y_pred_train_svr = best_svr.predict(X_train)
# ------------------------------------------------------------
# fazendo a predicao no conjunto de teste
y_pred_test_svr = best_svr.predict(X_test)
# ------------------------------------------------------------
# avaliando o desempenho, coeficiente de determinacao (R²), do conjunto de treinamento
r2_train_svr = r2_score(y_train, y_pred_train_svr)
print(f"R² Score no conjunto de treinamento (Modelo SVR): {r2_train_svr:.4f}")
# o Coeficiente de Determinação (R²) do conjunto de treinamento do modelo de SVR foi de 99,09%
# ------------------------------------------------------------
# avaliando o desempenho, coeficiente de determinacao (R²), do conjunto de teste
r2_test_svr = r2_score(y_test, y_pred_test_svr)
print(f"R² Score no conjunto de teste (Modelo SVR): {r2_test_svr:.4f}")
# o Coeficiente de Determinação (R²) do conjunto de treinamento do modelo de SVR foi de 96,83%
```

Os melhores hiperparâmetros encontrados foram:

- Kernel - escolhe como os dados serão processados: linear
- C: 1000 - valores altos de C indica que o modelo dá mais importância a minimizar o erro
- ε - configura a margem de tolerância para erros pequenos (largura do tubo): 0.5

Os coeficientes de determinação foram:

- Conjunto de treinamento: R^2 = 99,09%
- Conjunto de teste: R^2 = 96,83%

SVR - Support Vector Regression

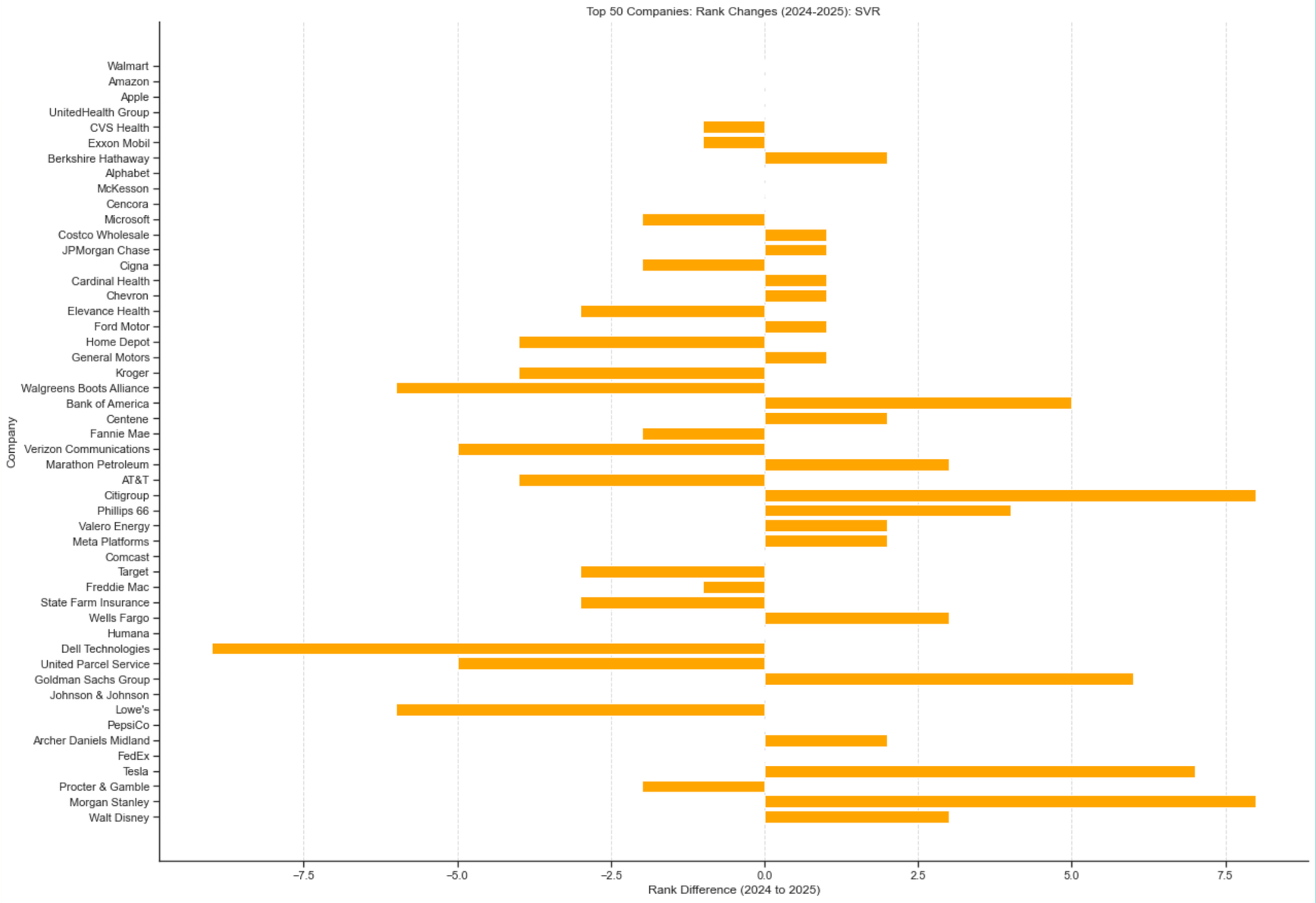
Por fim, fazemos a inferência com o modelo para o ano de 2025, ou seja, utilizamos o modelo previamente treinado com dados históricos. Para a inferência utilizamos os anos de 2021-2024.

```python
# ---------------------------------------------------------------
# inferencia com o modelo: SVR
# ---------------------------------------------------------------
# inferencia para o ano de 2025
y_2025_svr = best_svr.predict(X_inf)
# convertendo para DataFrame
df_2025_svr = pd.DataFrame(y_2025_svr)
df_2025_svr['company'] = ft_completo['company'] # adiciona a coluna 'company'
df_2025_svr = df_2025_svr.rename(columns = {0: 'revenues_2025'}) # renomeia a coluna do faturamento de 2025
# colocando a coluna 'company' a esquerda no DataFrame da coluna do faturamento predito de 2025
cols = ['company'] + [col for col in df_2025_svr.columns if col != 'company']
df_2025_svr = df_2025_svr[cols]
# criando a coluna de 'rank'
df_2025_svr['rank'] = df_2025_svr['revenues_2025'].rank(ascending = False, method = 'min')  # cria o rank com base em 'revenues_2025'
# colocando a coluna 'rank' mais a esquerda no DataFrame
cols = ['rank'] + [col for col in df_2025_svr.columns if col != 'rank']
df_2025_svr = df_2025_svr[cols]
# visualizando o DataFrame
print(df_2025_svr)
```

Índice	rank	company	revenues_2025	rank_difference	revenue_difference
0	1	Walmart	703853	0	55727.8
1	2	Amazon	608365	0	33579.5
2	3	Apple	446580	0	63295.1
3	4	UnitedHealth Group	383795	0	12173
5	5	CVS Health	369199	-1	11423.3
6	6	Exxon Mobil	348539	-1	3957.01
4	7	Berkshire Hathaway	333001	2	-31480.6
7	8	Alphabet	314954	0	7560.34
8	9	McKesson	296343	0	19632.5
9	10	Cencora	268263	0	6089.94
12	11	Microsoft	257380	-2	45465
10	12	Costco Wholesale	257093	1	14802.9
11	13	JPMorgan Chase	220748	1	-18677
15	14	Cigna	210027	-2	14761.9
13	15	Cardinal Health	207601	1	2589.4
14	16	Chevron	203174	1	2225.09
19	17	Elevance Health	176824	-3	5483.77
16	18	Ford Motor	174562	1	-1629
22	19	Home Depot	173114	-4	20445.2
18	20	General Motors	171651	1	-190.774
24	21	Kroger	165485	-4	15446.2
27	22	Walgreens Boots Alliance	164302	-6	25221.3
17	23	Bank of America	161888	5	-10024.4
21	24	Centene	158222	2	4222.87
26	25	Fannie Mae	157799	-2	16559.4
30	26	Verizon Communications	155488	-5	21513.9
23	27	Marathon Petroleum	153344	3	3036.59
31	28	AT&T	149501	-4	27073
20	29	Citigroup	148124	8	-8696.22

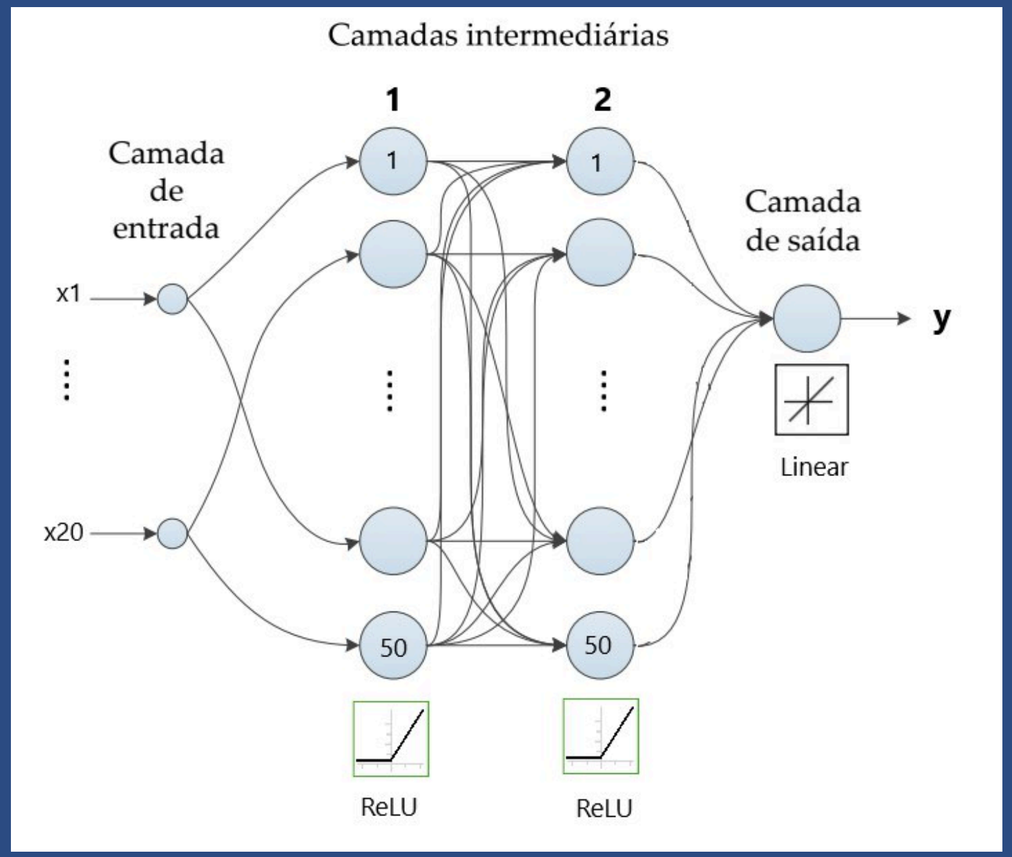
SVR - Support Vector Regression

Mudança no ranking de 2024 para 2025 das 50 empresas do topo da lista (795 empresas no total).



MLPRegression - Multi-Layer Perceptron Regressor

O MLP Regression (perceptron multicamadas para regressão) é um tipo de rede neural artificial organizada em camadas, começando com a camada de entrada, onde os dados são introduzidos, seguido por camadas ocultas onde as computações são realizadas usando uma função de ativação e, finalmente, a camada de saída onde as previsões ou decisões são feitas.




```python
# ------------------------------------------------------------

# MLPRegressor - Multi-Layer Perceptron Regressor

# ------------------------------------------------------------

# # definindo o modelo MPLRegressor

mlp = MLPRegressor(max_iter = 1000, random_state = 42)

# ------------------------------------------------------------



# definindo o grid de hiperparametros a ser buscado

param_grid = {
    'hidden_layer_sizes': [(50,), (100,), (50, 50)],
    'activation': ['relu', 'tanh'],
    'solver': ['adam', 'sgd'],
    'alpha': [0.0001, 0.001, 0.01],
    'learning_rate': ['constant', 'adaptive']
}

# ------------------------------------------------------------

# Grid Search com validacao cruzada

grid_search = GridSearchCV(
    estimator = mlp,
    param_grid = param_grid,
    scoring = 'neg_mean_squared_error',  # minimiza o erro quadratico medio
    cv = 10, # numero de folds para validacao cruzada
    verbose = 2,
    n_jobs = -1  # utiliza todos os nucleos disponiveis
)

# ------------------------------------------------------------

# treinando o modelo com os diferentes hiperparametros para chegar na melhor selecao de hiperparametros

grid_search.fit(X_train, y_train)

# ------------------------------------------------------------

# melhores hiperparametros encontrados pelo Grid Search

best_params = grid_search.best_params_

print("Melhores parâmetros encontrados pelo Grid Search:", best_params)

# melhores hiperparametros encontrados: {'activation': 'relu', 'alpha': 0.0001, 'hidden_layer_sizes': (50, 50), 'learning_rate': 'constant', 'solver': 'adam'}
```

MLPRegression - Multi-Layer Perceptron Regressor

Depois de treinar a MLP com os melhores hiperparâmetros encontrados pelo Grid Search (busca em grade), calculamos o coeficiente de determinação (R^2) dos conjuntos de treinamento e de teste, que é a métrica estatística que avalia o desempenho do modelo.

```python
# modelo final com os melhores hiperparametros

best_mlp = MLPRegressor(
    hidden_layer_sizes = best_params['hidden_layer_sizes'],
    activation = best_params['activation'],
    solver = best_params['solver'],
    alpha = best_params['alpha'],
    learning_rate = best_params['learning_rate'],
    max_iter = 20000,
    random_state = 42
)

# ---------------------------------------------------------------

# treinando o modelo com os melhores hiperparametros

best_mlp.fit(X_train, y_train)

# ---------------------------------------------------------------

# fazendo a predicao no conjunto de treinamento

y_pred_train_mlp = best_mlp.predict(X_train)

# ---------------------------------------------------------------

# fazendo a predicao no conjunto de teste

y_pred_test_mlp = best_mlp.predict(X_test)

# ---------------------------------------------------------------

# avaliando o desempenho, coeficiente de determinacao (R²), do conjunto de treinamento

r2_train_mlp = r2_score(y_train, y_pred_train_mlp)

print(f"R² Score no conjunto de treinamento (Modelo MLP): {r2_train_mlp:.4f}")

# o Coeficiente de Determinação (R²) do conjunto de treinamento do modelo de MLPRegressor foi de 99,88%

# ---------------------------------------------------------------

# avaliando o desempenho, coeficiente de determinacao (R²), do conjunto de teste

r2_test_mlp = r2_score(y_test, y_pred_test_mlp)

print(f"R² Score no conjunto de teste (Modelo MLP): {r2_test_mlp:.4f}")

# o Coeficiente de Determinação (R²) do conjunto de treinamento do modelo de MLPRegressor foi de 97,91%
```

Os melhores hiperparâmetros encontrados foram:

- Estrutura das camadas ocultas: duas camadas com 50 neurônios cada.
- Função de ativação 'relu' (Rectified Linear Unit).
- Otimizador usado para ajustar os pesos da rede: 'adam'.
- Peso de penalidade (alpha): 0.0001 (valores pequenos permitem maior flexibilidade no ajuste dos pesos).
- Taxa de aprendizado fixa durante todo o treinamento: 'constant'.

Os coeficientes de determinação foram:

- Conjunto de treinamento: R^2 = 99,88%
- Conjunto de teste: R^2 = 97,91%

MLPRegression - Multi-Layer Perceptron Regressor

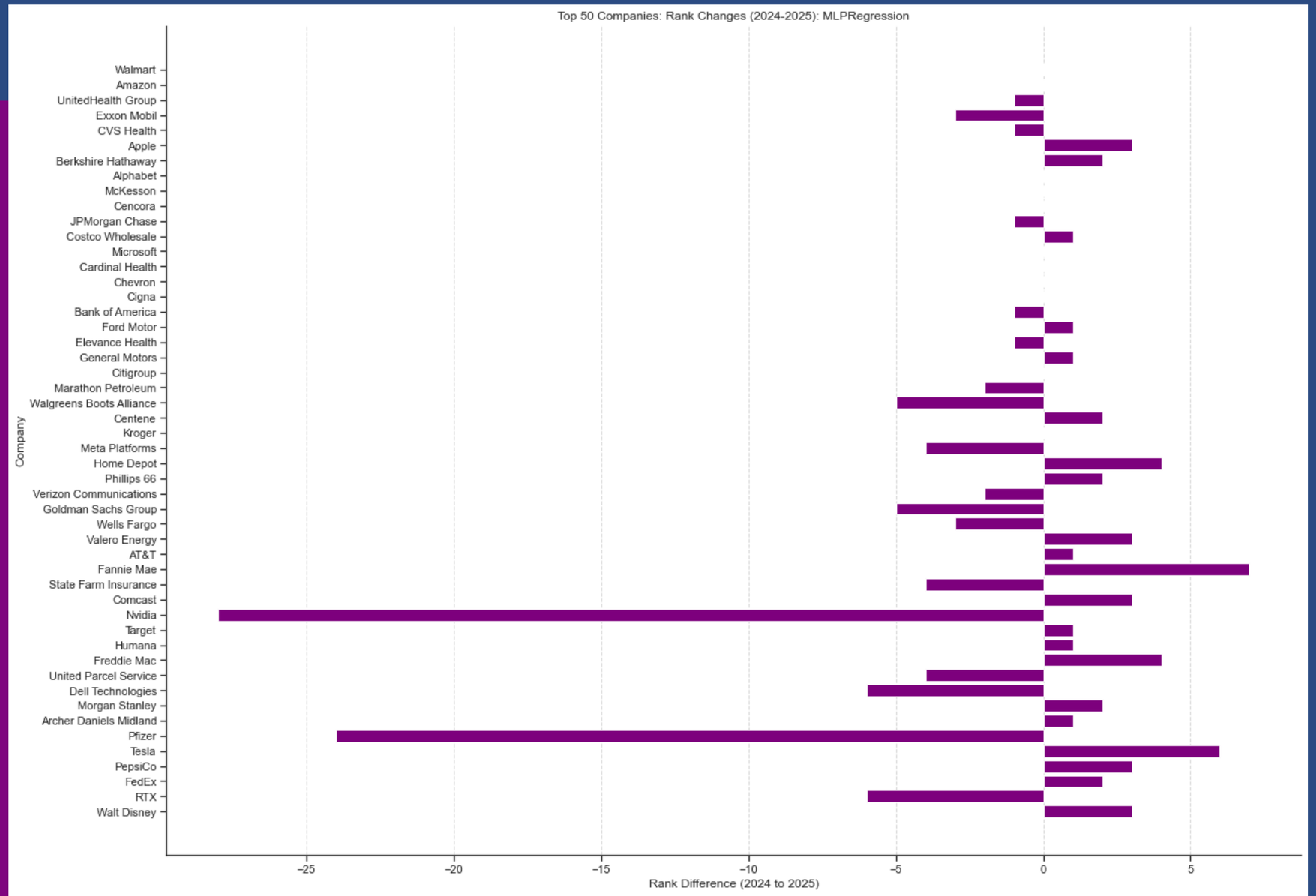
Por fim, fazemos a inferência com o modelo para o ano de 2025, ou seja, utilizamos o modelo previamente treinado com dados históricos. Para a inferência utilizamos os anos de 2021-2024.

```python
# ---------------------------------------------------------------
# inferencia com o modelo: MLPRegressor
# ---------------------------------------------------------------
# inferencia para o ano de 2025
y_2025_MLP = best_mlp.predict(X_inf)
# convertendo para DataFrame
df_2025_MLP = pd.DataFrame(y_2025_MLP)
df_2025_MLP['company'] = ft_completo['company'] # adiciona a coluna 'company'
df_2025_MLP = df_2025_MLP.rename(columns = {0: 'revenues_2025'}) # renomeia a coluna do faturamento de 2025
# colocando a coluna 'company' a esquerda no DataFrame da coluna do faturamento predito de 2025
cols = ['company'] + [col for col in df_2025_MLP.columns if col != 'company']
df_2025_MLP = df_2025_MLP[cols]
# criando a coluna de 'rank'
df_2025_MLP['rank'] = df_2025_MLP['revenues_2025'].rank(ascending = False, method = 'min')  # cria o rank com base em 'revenues_2025'
# colocando a coluna 'rank' mais a esquerda no DataFrame
cols = ['rank'] + [col for col in df_2025_MLP.columns if col != 'rank']
df_2025_MLP = df_2025_MLP[cols]
# visualizando o DataFrame
print(df_2025_MLP)
```

Índice	rank	company	revenues_2025	rank_difference	revenue_difference
0	1	Walmart	717152	0	69026.8
1	2	Amazon	648928	0	74143.5
3	3	UnitedHealth Group	424228	-1	52606.4
6	4	Exxon Mobil	416571	-3	71988.7
5	5	CVS Health	400909	-1	43133.3
2	6	Apple	380749	3	-2536.3
4	7	Berkshire Hathaway	361029	2	-3452.81
7	8	Alphabet	305423	0	-1970.55
8	9	McKesson	304848	0	28137.5
9	10	Cencora	292329	0	30155.3
11	11	JPMorgan Chase	287340	-1	47914.5
10	12	Costco Wholesale	277115	1	34825.2
12	13	Microsoft	246296	0	34381
13	14	Cardinal Health	221377	0	16365
14	15	Chevron	220367	0	19417.5
15	16	Cigna	216706	0	21440.7
17	17	Bank of America	215290	-1	43377.6
16	18	Ford Motor	196130	1	19938.8
19	19	Elevance Health	185545	-1	14204.5
18	20	General Motors	184897	1	13054.8
20	21	Citigroup	184069	0	27248.9
23	22	Marathon Petroleum	172277	-2	21970
27	23	Walgreens Boots Alliance	171212	-5	32130.8
21	24	Centene	164845	2	10845.7
24	25	Kroger	164668	0	14629.4
29	26	Meta Platforms	155124	-4	20221.6
22	27	Home Depot	153575	4	906.203
25	28	Phillips 66	151915	2	2025.24

MLPRegression - Multi-Layer Perceptron Regressor

Mudança no ranking de 2024 para 2025 das 50 empresas do topo da lista (795 empresas no total).



SVR X MLPRegression

Índice	rank_svr	rank_MLP	company	revenues_2025_svr	revenues_2025_MLP	rank_difference	revenue_difference
0	1	1	Walmart	703853	717152	0	-13299
1	2	2	Amazon	608365	648928	0	-40563.9
2	3	6	Apple	446580	380749	-3	65831.5
3	4	3	UnitedHealth Group	383795	424228	1	-40433.4
5	5	5	CVS Health	369199	400909	0	-31710
6	6	4	Exxon Mobil	348539	416571	2	-68031.6
4	7	7	Berkshire Hathaway	333001	361029	0	-28027.8
7	8	8	Alphabet	314954	305423	0	9530.89
8	9	9	McKesson	296343	304848	0	-8504.98
9	10	10	Cencora	268263	292329	0	-24065.3
12	11	13	Microsoft	257380	246296	-2	11084
10	12	12	Costco Wholesale	257093	277115	0	-20022.3
11	13	11	JPMorgan Chase	220748	287340	2	-66591.6
15	14	16	Cigna	210027	216706	-2	-6678.8
13	15	14	Cardinal Health	207601	221377	1	-13775.5
14	16	15	Chevron	203174	220367	1	-17192.4
19	17	19	Elevance Health	176824	185545	-2	-8720.76
16	18	18	Ford Motor	174562	196130	0	-21567.8
22	19	27	Home Depot	173114	153575	-8	19539
18	20	20	General Motors	171651	184897	0	-13245.6
24	21	25	Kroger	165485	164668	-4	816.799
27	22	23	Walgreens Boots Alliance	164302	171212	-1	-6909.57
17	23	17	Bank of America	161888	215290	6	-53402
21	24	24	Centene	158222	164845	0	-6622.85
26	25	34	Fannie Mae	157799	131813	-9	25986
30	26	29	Verizon Communications	155488	148727	-3	6760.69
23	27	22	Marathon Petroleum	153344	172277	5	-18933.4
31	28	33	AT&T	149501	134598	-5	14903.3
20	29	21	Citigroup	148124	184069	8	-35945.1

Árvore de Regressão

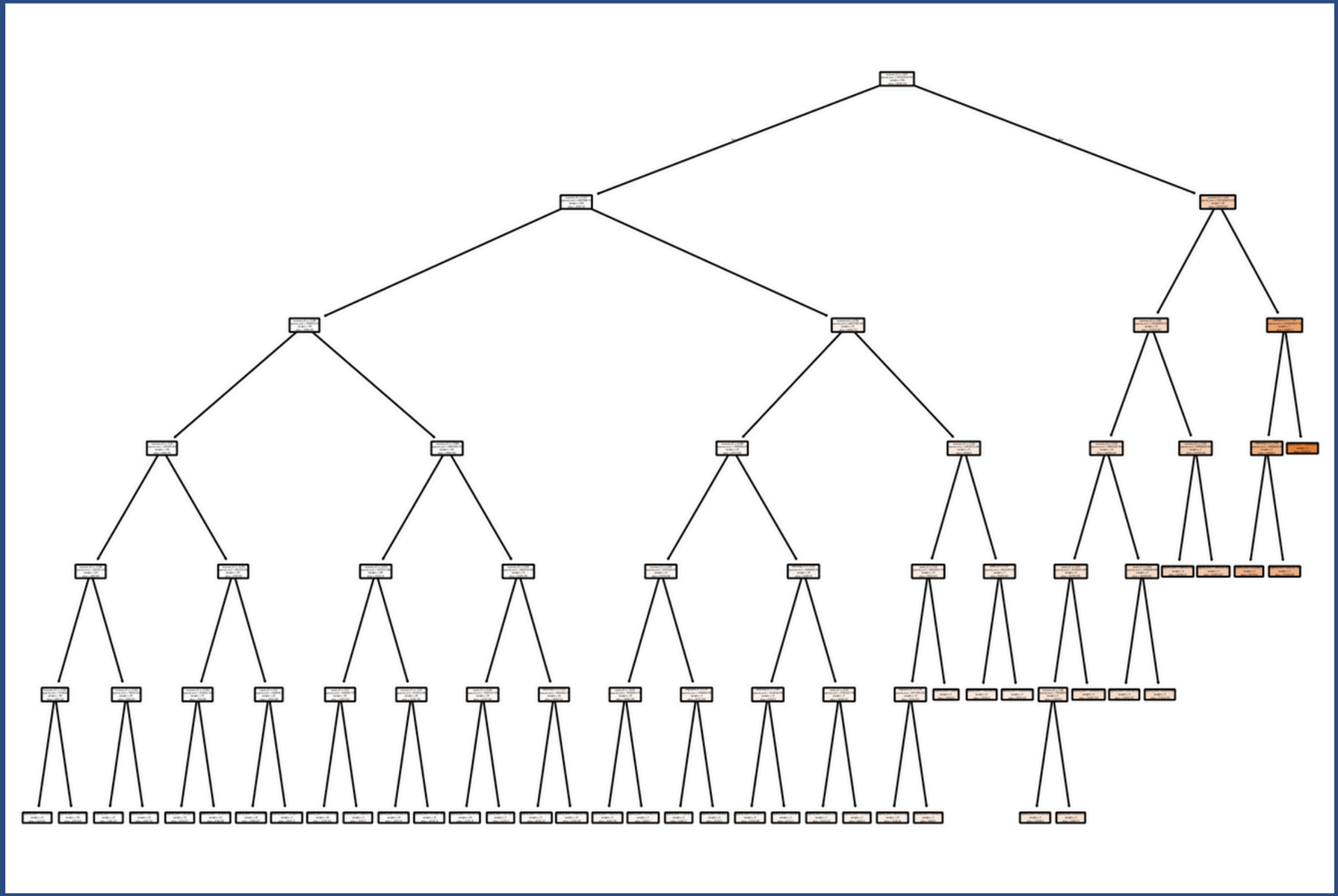
```python

# ----------------------------------------------------------------

# Regression Tree - Árvore de Regressão

# ----------------------------------------------------------------

# definindo o modelo regtree

regtree = DecisionTreeRegressor(criterion = 'squared_error', random_state = 42, max_depth

# ----------------------------------------------------------------

# definindo o grid de hiperparametros a ser buscado

parameters = {
    'max_depth': [3, 4, 5, 6, 7],
    'ccp_alpha': [0.01, 0.005, 0.001],
    'min_samples_split': range(2, 15, 1),
    'min_samples_leaf': range(2, 15, 1)
}

# ----------------------------------------------------------------

# Grid Search com validacao cruzada

gs_regtree = GridSearchCV(
    estimator = regtree,
    param_grid = parameters,
    cv = 10,  # numero de folds para validacao cruzada
    scoring = 'r2',  # metrica de avaliacao: coeficiente de determinacao
    n_jobs = -1,  # paralelizacao
    verbose = 1  # nivel de detalhamento
)
```

Árvore de Regressão

```python
# ----------------------------------------------------------------

# melhores hiperparametros encontrados pelo Grid Search

print("Melhores parâmetros encontrados:", gs_regtree.best_params_)

# Melhores parâmetros encontrados: {'ccp_alpha': 0.01, 'max_depth': 6, 'min_samples_leaf': 2, 'min_samples_split': 5}

# ----------------------------------------------------------------
```

```python
# ----------------------------------------------------------------

# avaliando o desempenho, coeficiente de determinacao (R²), do conjunto de treinamento

r2_train_regtree = r2_score(y_train, y_pred_train_regtree)

print(f"R² Score no conjunto de treinamento (Modelo Árvore de Regressão): {r2_train_regtree:.4f}")

# o Coeficiente de Determinação (R²) do conjunto de treinamento do modelo de Árvore de Regressão foi de 99,66%

# ----------------------------------------------------------------

# avaliando o desempenho, coeficiente de determinacao (R²), do conjunto de teste

r2_test_regtree = r2_score(y_test, y_pred_test_regtree)

print(f"R² Score no conjunto de teste (Modelo Árvore de Regressão): {r2_test_regtree:.4f}")

# o Coeficiente de Determinação (R²) do conjunto de treinamento do modelo de Árvore de Regressão foi de 94,08%

# ----------------------------------------------------------------
```

Árvore de Regressão



Profundidade da Árvore: 6

Número de folhas: 39

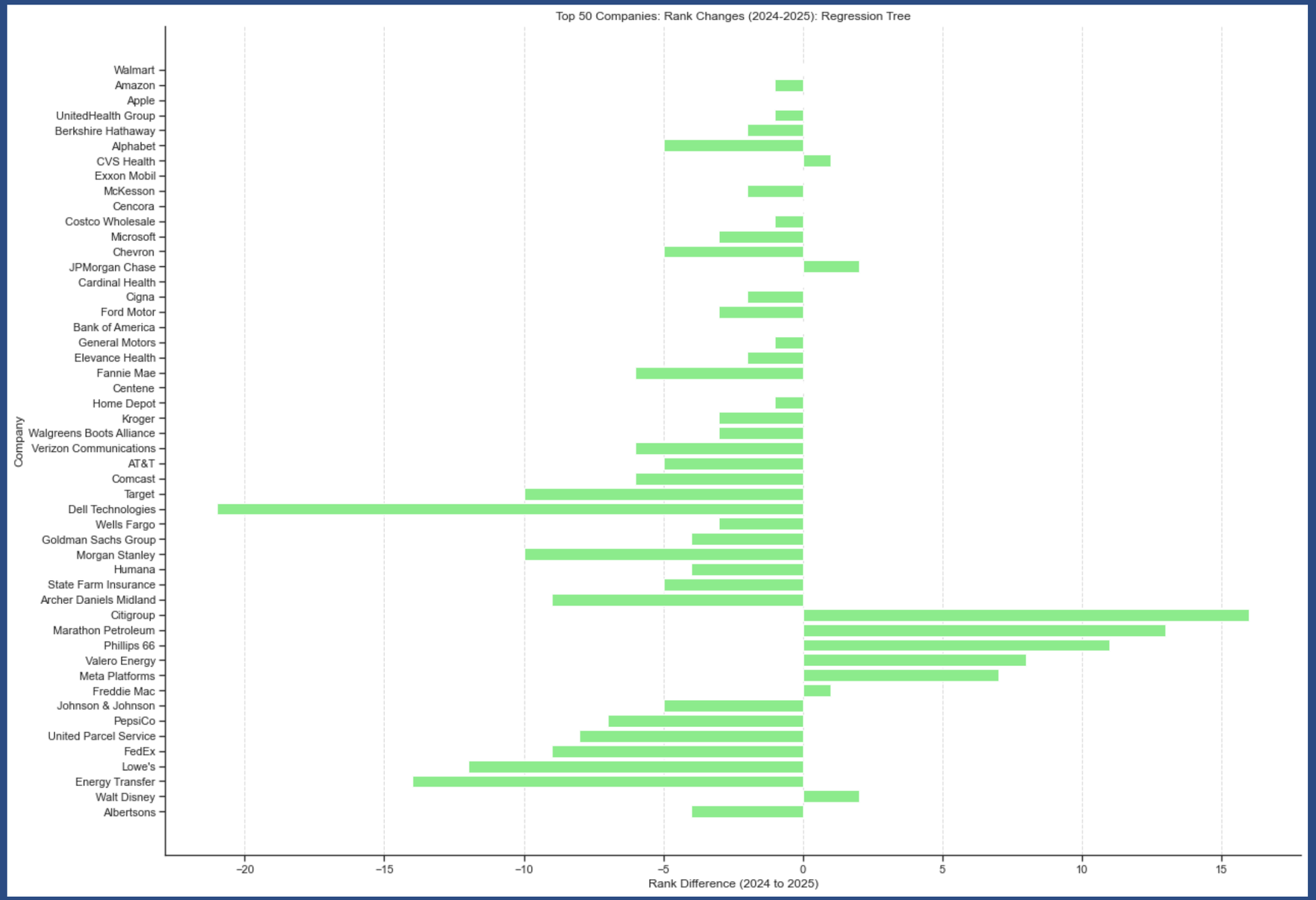
Árvore de Regressão

rank	company	revenues_2025	nk_differenc	revenue_difference
1	Walmart	611455	0	-36670
1	Amazon	611455	-1	36670
3	Apple	373130	0	-10155.3
3	UnitedHealth Group	373130	-1	1507.67
3	Berkshire Hathaway	373130	-2	8647.67
3	Alphabet	373130	-5	65735.7
7	CVS Health	351179	1	-6597
7	Exxon Mobil	351179	0	6597
7	McKesson	351179	-2	74468
10	Cencora	260391	0	-1781.93
10	Costco Wholesale	260391	-1	18101.5
10	Microsoft	260391	-3	48476.5
10	Chevron	260391	-5	59442.5
14	JPMorgan Chase	203285	2	-36139.8
14	Cardinal Health	203285	0	-1726.75
14	Cigna	203285	-2	8020.25
14	Ford Motor	203285	-3	27094.2
18	Bank of America	173124	0	1212.33
18	General Motors	173124	-1	1282.33

rank	company	revenues_2025	nk_differenc	revenue_difference
18	Elevance Health	173124	-2	1784.33
21	Fannie Mae	156657	-6	15417.3
22	Centene	150955	0	-3043.67
22	Home Depot	150955	-1	-1713.67
22	Kroger	150955	-3	916.333
25	Walgreens Boots Alliance	136528	-3	-2553.5
25	Verizon Communications	136528	-6	2553.5
27	AT&T	119780	-5	-2648
27	Comcast	119780	-6	-1792
27	Target	119780	-10	12368
27	Dell Technologies	119780	-21	31355
31	Wells Fargo	108234	-3	-7106
31	Goldman Sachs Group	108234	-4	-184
31	Morgan Stanley	108234	-10	12040
34	Humana	101602	-4	-4771.6
34	State Farm Insurance	101602	-5	-2596.2
34	Archer Daniels Midland	101602	-9	7667.4
37	Citigroup	90575.7	16	-66244.3
37	Marathon Petroleum	90575.7	13	-59731.3

rank	company	revenues_2025	nk_differenc	revenue_difference
31	Goldman Sachs Group	108234	-4	-184
31	Morgan Stanley	108234	-10	12040
34	Humana	101602	-4	-4771.6
34	State Farm Insurance	101602	-5	-2596.2
34	Archer Daniels Midland	101602	-9	7667.4
37	Citigroup	90575.7	16	-66244.3
37	Marathon Petroleum	90575.7	13	-59731.3
37	Phillips 66	90575.7	11	-59314.3
37	Valero Energy	90575.7	8	-48425.3
37	Meta Platforms	90575.7	7	-44326.3
37	Freddie Mac	90575.7	1	-17474.3
37	Johnson & Johnson	90575.7	-5	-4619.33
37	PepsiCo	90575.7	-7	-895.333
37	United Parcel Service	90575.7	-8	-382.333
37	FedEx	90575.7	-9	420.667
37	Lowe's	90575.7	-12	4198.67
37	Energy Transfer	90575.7	-14	11989.7
49	Walt Disney	90184.5	2	1286.5
49	Albertsons	90184.5	-4	12534.8

Árvore de Regressão



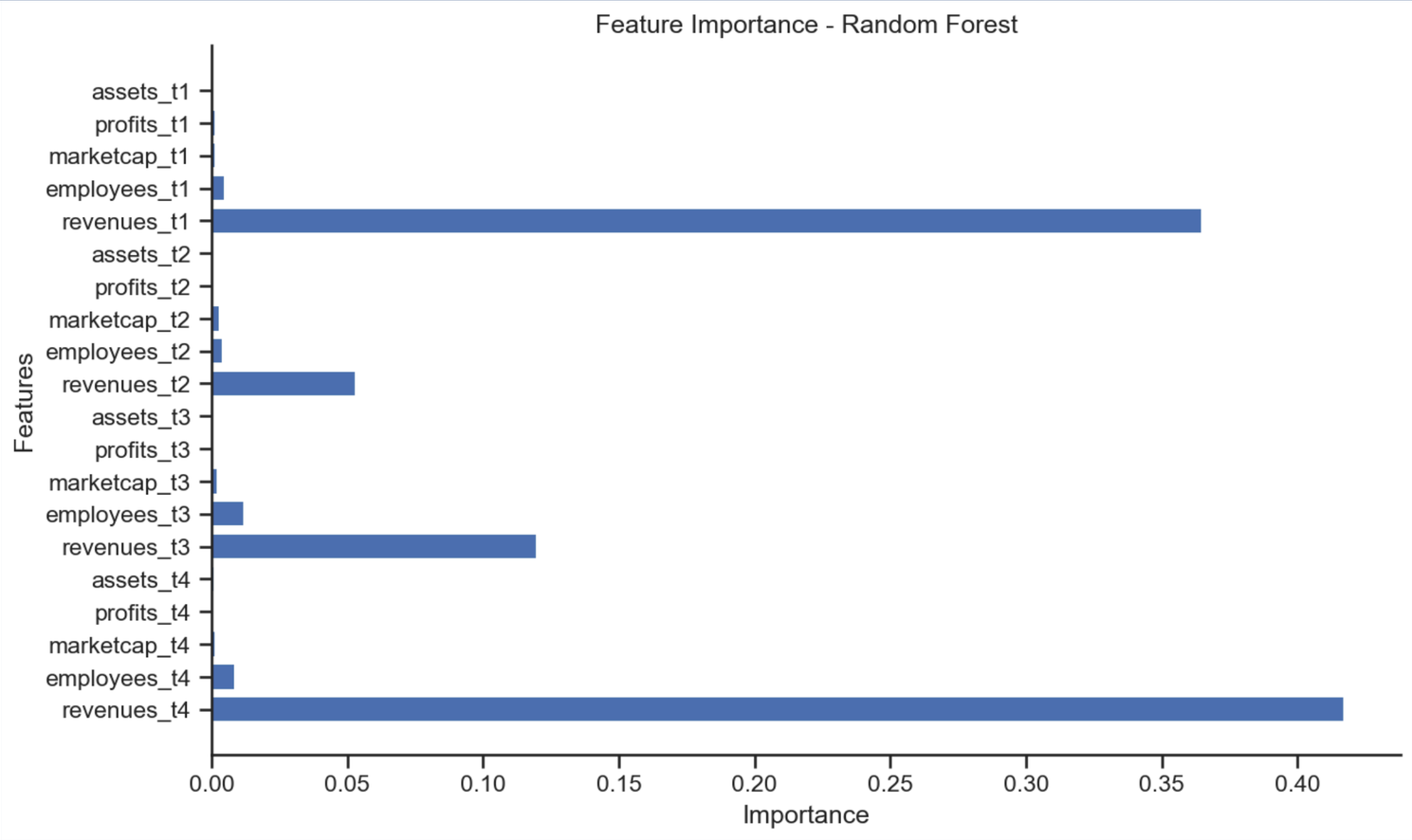
Random Forest

```
Best parameters found: {'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}
```

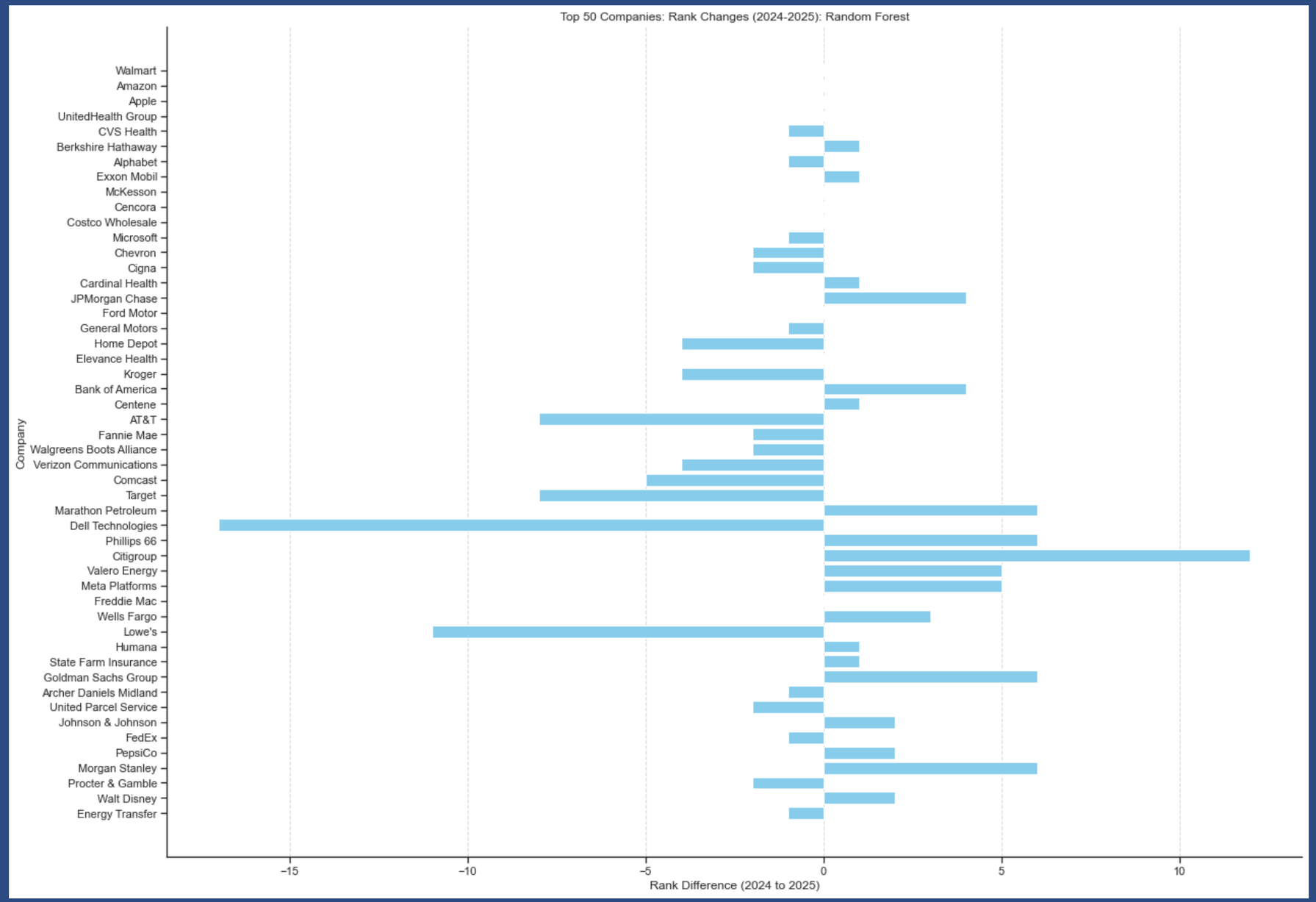
```
R² Score on training set: 0.9944
R² Score on test set: 0.9488
```

→

```
R² Score on training set: 0.9931
R² Score on test set: 0.9513
```



Random Forest



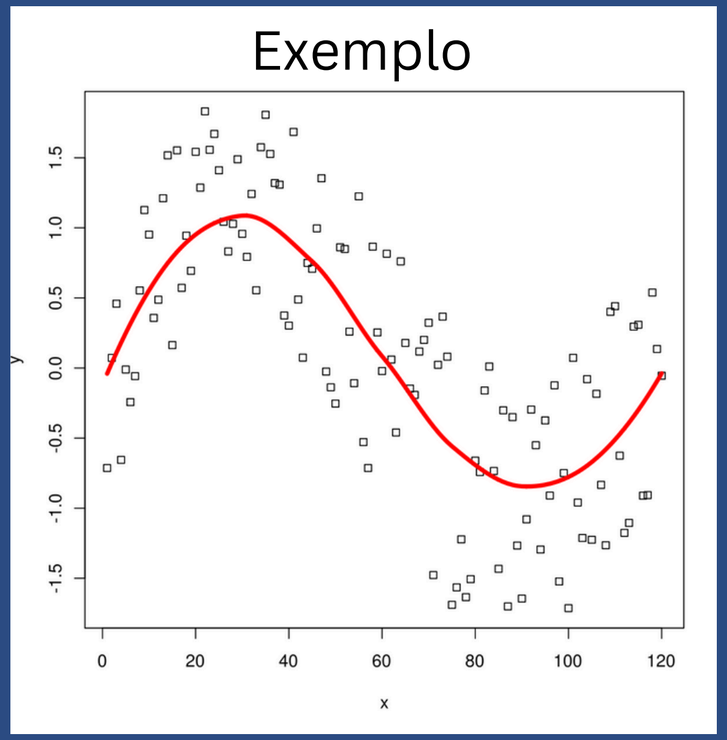
Random Forest

Top 50 Companies Rankings

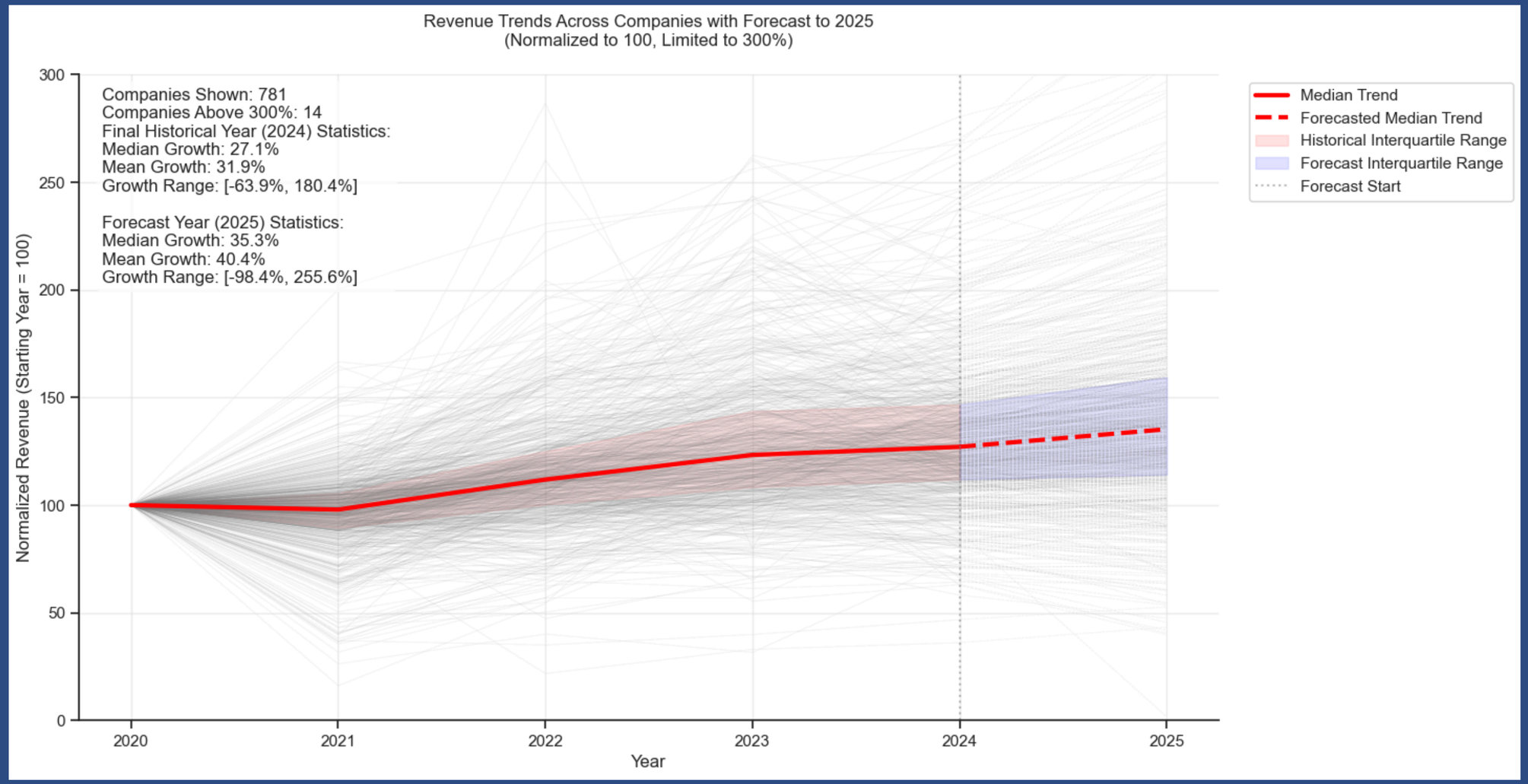
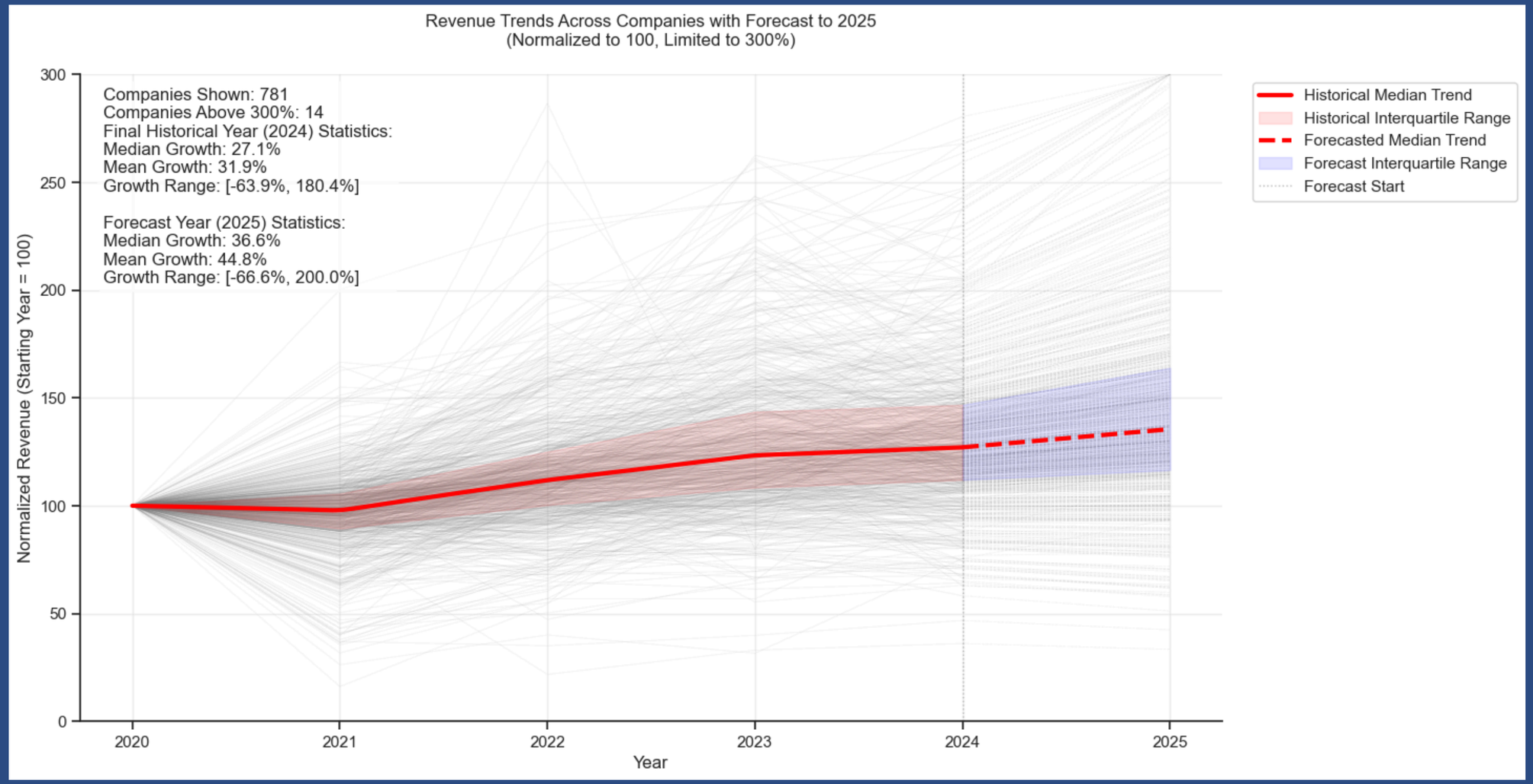
Walmart	1.0	1.0
Amazon	2.0	2.0
Apple	3.0	3.0
UnitedHealth Group	4.0	4.0
Berkshire Hathaway	5.0	6.0
CVS Health	6.0	5.0
Exxon Mobil	7.0	8.0
Alphabet	8.0	7.0
McKesson	9.0	9.0
Cencora	10.0	10.0
Costco Wholesale	11.0	11.0
JPMorgan Chase	12.0	16.0
Microsoft	13.0	12.0
Cardinal Health	14.0	15.0
Chevron	15.0	13.0
Cigna	16.0	14.0
Ford Motor	17.0	17.0
Bank of America	18.0	22.0
General Motors	19.0	18.0
Elevance Health	20.0	20.0
Citigroup	21.0	33.0
Centene	22.0	23.0
Home Depot	23.0	19.0
Marathon Petroleum	24.0	30.0
Kroger	25.0	21.0
Phillips 66	26.0	32.0
Fannie Mae	27.0	25.0
Walgreens Boots Alliance	28.0	26.0
Valero Energy	29.0	34.0
Meta Platforms	30.0	35.0
Verizon Communications	31.0	27.0
AT&T	32.0	24.0
Comcast	33.0	28.0
Wells Fargo	34.0	37.0
Goldman Sachs Group	35.0	41.0
Freddie Mac	36.0	36.0
Target	37.0	29.0
Humana	38.0	39.0
State Farm Insurance	39.0	40.0
Tesla	40.0	55.0
Morgan Stanley	41.0	47.0
Johnson & Johnson	42.0	44.0
Archer Daniels Midland	43.0	42.0
PepsiCo	44.0	46.0
United Parcel Service	45.0	43.0
FedEx	46.0	45.0
Walt Disney	47.0	49.0
Dell Technologies	48.0	31.0
Lowe's	49.0	38.0
Procter & Gamble	50.0	48.0

Regressão Lowess

company	2020	2021	2022	2023	2024
Walmart	523964	559151.0	572754.0	611289	648125.0
Amazon	280522	386064.0	469822.0	513983	574785.0
Apple	260174	274515.0	365817.0	394328	383285.0
UnitedHealth Group	242155	257141.0	287597.0	324162	371622.0
Berkshire Hathaway	254616	245510.0	276094.0	302089	364482.0
...	...	...	...	...	...
Nordson	2194	2121.1	2362.2	2590	2628.6
Summit Materials	2222	2332.5	2409.7	2413	2619.5
Valvoline	2390	2353.0	2981.0	3931	2617.9
Louisiana-Pacific	2310	2788.0	4553.0	4309	2581.0
Belden	2492	1971.9	2408.1	2619	2512.1



Regressão Lowess



Regressão Kernel